

```
def isSafe(mat, r, c):  
  
    # Check column  
    for i in range(r):  
        if mat[i][c] == 'Q':  
            return False  
  
    # Check upper left diagonal  
    i, j = r, c  
    while i >= 0 and j >= 0:  
        if mat[i][j] == 'Q':  
            return False  
        i -= 1  
        j -= 1  
  
    # Check upper right diagonal  
    i, j = r, c  
    while i >= 0 and j < len(mat):  
        if mat[i][j] == 'Q':  
            return False  
        i -= 1  
        j += 1  
  
    return True
```

Commands + Code + Text ▶ Run all

To exit full screen, press and hold Esc

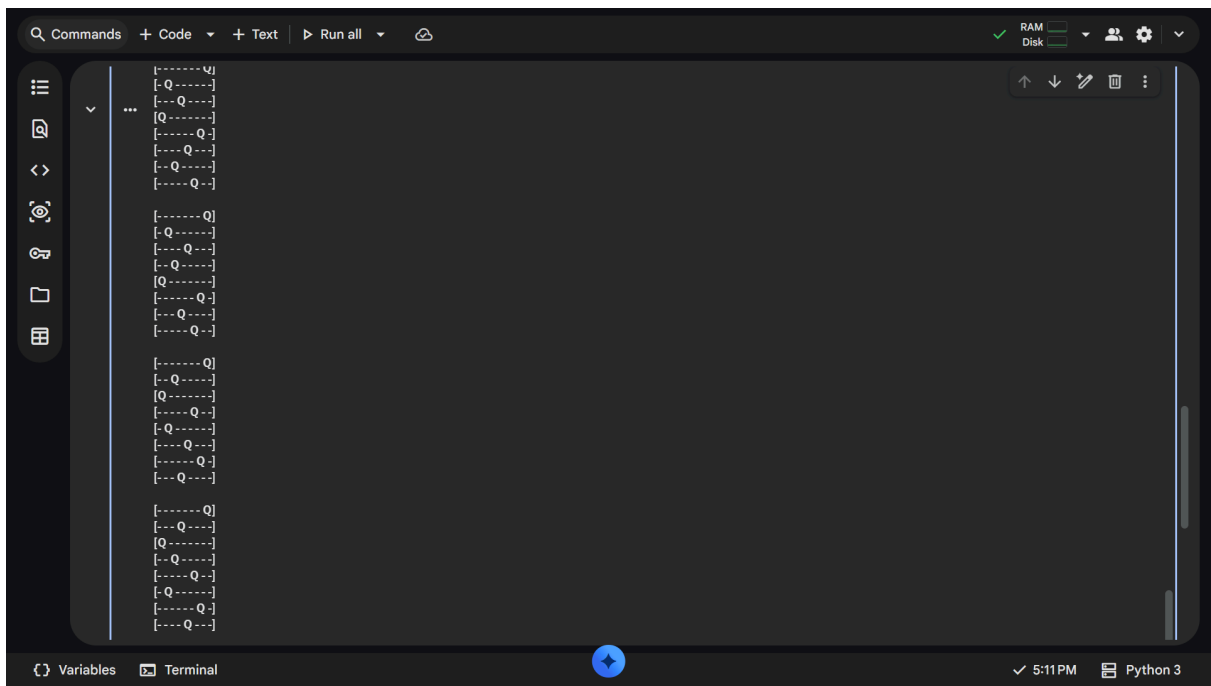
RAM
Disk

[1] ✓ 5s

```
def printSolution(mat):  
  
    for row in mat:  
        print(str(row).replace(',', '').replace(' ', ''))  
  
    print()  
  
def nQueen(mat, r):  
  
    # Base case  
    if r == len(mat):  
        printSolution(mat)  
        return  
  
    # Try placing queen in each column  
    for i in range(len(mat)):  
  
        if isSafe(mat, r, i):  
  
            # Place queen  
            mat[r][i] = 'Q'  
  
            # Recur for next row  
            nQueen(mat, r + 1)  
  
            # Backtrack  
            mat[r][i] = ''
```

Variables Terminal

5:11 PM Python 3



```
import networkx as nx
import matplotlib.pyplot as plt

# FUNCTION TO SOLVE DFS
def solveDFS(graph, v, visited):

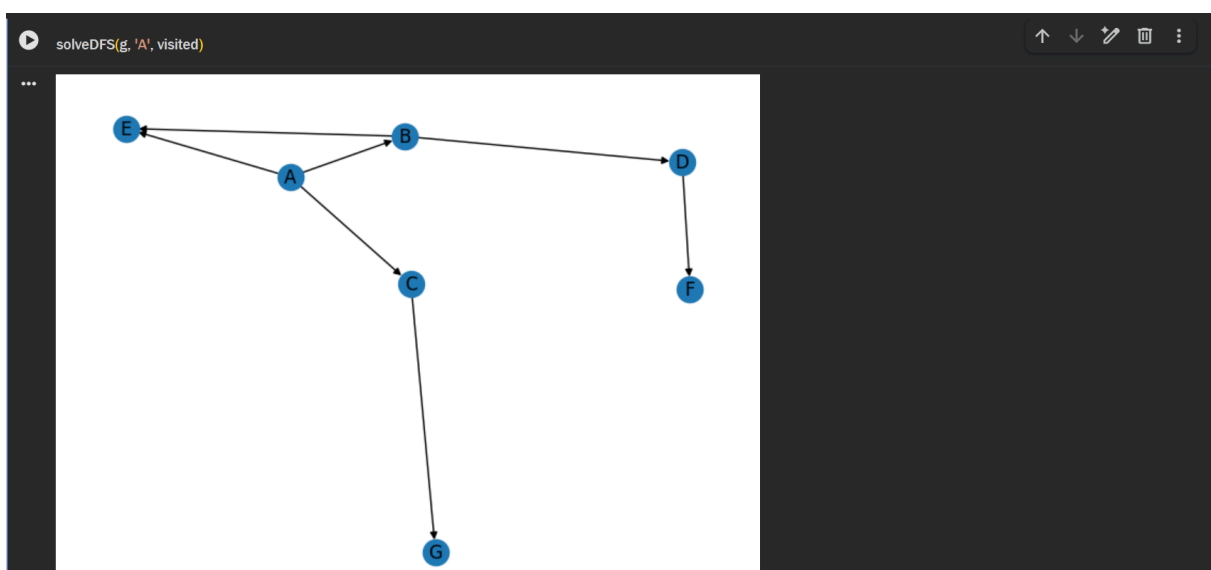
    visited.add(v)
    print(v, end=' ')

    for neighbour in graph[v]:

        if neighbour not in visited:
            solveDFS(graph, neighbour, visited)

# Create a graph using NetworkX
g = nx.DiGraph()

# Add edges for the graph
g.add_edges_from([
    ('A', 'B'),
    ('A', 'C'),
    ('C', 'G'),
```



```
from collections import deque

def DFS(a, b, target):

    m = {}
    isSolvable = False
    path = []

    q = deque()
    q.append((0, 0))

    while len(q) > 0:

        u = q.popleft()

        # Skip already visited states
        if (u[0], u[1]) in m:
            continue

        # Skip invalid states
        if (u[0] > a or u[1] > b or u[0] < 0 or u[1] < 0):
            continue
```

```
        # Target found
        if u[0] == target or u[1] == target:

            isSolvable = True

            if u[0] == target:

                if u[1] != 0:
                    path.append([u[0], 0])

            else:

                if u[0] != 0:
                    path.append([0, u[1]])

            # Print solution path
            sz = len(path)

            for i in range(sz):
                print(" ", path[i][0], " ", path[i][1], " ")

            break

        # Fill Jug2
        q.append([u[0], b])

        # Fill Jug1
        q.append([a, u[1]])
```

```
q.append([a, 0])

# Empty Jug1
q.append([0, b])

if not isSolvable:
    print("No solution")

Jug1 = 4
Jug2 = 3
target = 2

print("Path from initial state to solution state ::")

DFS(Jug1, Jug2, target)

... Path from initial state to solution state ::
( 0 , 0 )
( 0 , 3 )
( 4 , 0 )
( 4 , 3 )
( 3 , 0 )
( 1 , 3 )
( 3 , 3 )
( 4 , 2 )
( 0 , 2 )
```

```
from math import inf as infinity
from random import choice
import platform
import time
from os import system
```

```
HUMAN = -1
COMP = +1
```

```
board = [
    [0, 0, 0],
    [0, 0, 0],
    [0, 0, 0]
]
```

```
def evaluate(state):
```

```
    if wins(state, COMP):
        score = +1
```

```
    elif wins(state, HUMAN):
        score = -1
```

```
    else:
        score = 0
```

```
    return wins(state, HUMAN) or wins(state, COMP)
```

```
def empty_cells(state):
```

```
    cells = []
```

```
    for x, row in enumerate(state):
```

```
        for y, cell in enumerate(row):
```

```
            if cell == 0:
                cells.append([x, y])
```

```
    return cells
```

```
def valid_move(x, y):
```

```
    if [x, y] in empty_cells(board):
        return True
```

```
    return False
```

```
def set_move(x, y, player):
```

```
    exit()
```

```
    except (KeyError, ValueError):
```

```
        print("Bad choice")
```

```
    while len(empty_cells(board)) > 0 and not game_over(board):
```

```
        if first == 'N':
```

```
            ai_turn(c_choice, h_choice)
```

```
            first = "
```

```
            human_turn(c_choice, h_choice)
```

```
            ai_turn(c_choice, h_choice)
```

```
    if wins(board, HUMAN):
```

```
        clean()
```

```
        print(f"Human turn [{h_choice}]")
```

```
        render(board, c_choice, h_choice)
```

```
        print("YOU WIN!")
```

$$\begin{array}{r} \text{---} \\ | \times | | \quad | | \quad | \\ \text{---} \\ | \quad | | \quad | | \quad | \\ \text{---} \\ | \quad | | \quad | | \quad | \end{array}$$

from collections import deque

Nodes to be explored

```
'C': [('D', 12)]
```

```
return None

# Graph definition
adjac_lis = {

    'A': [('B', 1), ('C', 3), ('D', 7)],

    'B': [('D', 5)],

    'C': [('D', 12)]

}

graph1 = Graph(adjac_lis)

graph1.a_star_algorithm('A', 'D')

... Path found: ['A', 'B', 'D']
['A', 'B', 'D']
```

```
class AStar:

    def __init__(self, heuristics, conditions, weight=1):

        self.H = heuristics
        self.conditions = conditions
        self.weight = weight

        self.parent = {}

        self.status = {} # 0 = unsolved, 1 = solved

        self.solution_graph = {}

    def get_node_status(self, node):

        return self.status.get(node, 0)

    def set_node_status(self, node, status):

        self.status[node] = status

    def get_heuristics(self, node):

        return self.H.get(node, 0)
```

```
if v in self.conditions:

    cond = self.conditions[v]

    # AND condition
    if 'AND' in cond:

        and_nodes = cond['AND']

        and_cost = sum(
            self.get_heuristics(node) + self.weight
            for node in and_nodes
        )

        cost_to_child_nodes_dict['AND'] = (
            and_cost,
            and_nodes
        )

    # OR condition
    if 'OR' in cond:

        or_nodes = cond['OR']

        for node in or_nodes:
```

```

    print("EXPANDING NODE: {v}")

    if self.get_node_status(v) >= 0:

        minimum_cost_dict = self.compute_minimum_cost_child_nodes(v)

        best_path_type = None
        min_cost = float("inf")
        best_nodes = []

        # Check AND path
        if minimum_cost_dict['AND']:

            min_cost, best_nodes = minimum_cost_dict['AND']

            best_path_type = 'AND'

        # Check OR paths
        for or_cost, or_nodes in minimum_cost_dict['OR']:

            if or_cost < min_cost:

                min_cost = or_cost

                best_nodes = or_nodes

            best_path_type = 'OR'

```

```

    self.H[v] = min_cost

    self.solution_graph[v] = best_nodes

    # Check whether all children solved
    solved = True

    for child in best_nodes:

        self.parent[child] = v

        if self.get_node_status(child) != 1:

            solved = False

    if solved:

        self.set_node_status(v, 1)

    # Backtracking
    if v != root_node:

        self.ao_star(self.parent[v], True)

    # Expand children
    if not back_tracking:

```

```

    'OR': ['G'],
    'AND': ['H', 'I']
},

'D': {
    'OR': ['J']
}
}

root_node = 'A'

solver = AStar(h, conditions, weight=1)

solver.ao_star(root_node, False)

print("\nFinal Heuristics:", solver.H)

print("\nSolution Graph:", solver.solution_graph)

```

... EXPANDING NODE: A
EXPANDING NODE: B
EXPANDING NODE: A
EXPANDING NODE: E
EXPANDING NODE: B
EXPANDING NODE: A

Final Heuristics: {'A': 8, 'B': 8, 'C': 2, 'D': 4, 'E': 7, 'F': 9, 'G': 3, 'H': 0, 'I': 0, 'J': 0}
Solution Graph: {'A': ['C', 'D'], 'B': ['E']}

```

# Rules: IF condition THEN result
rules = [
    (["A", "B"], "C"),
    (["C"], "D"),
    (["D"], "E")
]
facts = ["A", "B"] # Initial facts
goal = "E"
def forward_chaining(rules, facts, goal):
    inferred = set(facts)
    while True:
        new_inferred = False
        for conditions, result in rules:
            if all(cond in inferred for cond in conditions):
                if result not in inferred:
                    print(f"Inferred: {result}")
                    inferred.add(result)
                    new_inferred = True
                if result == goal:
                    return True
        if not new_inferred:
            return False

```

```

        inferred.add(result)
        new_inferred = True
        if result == goal:
            return True
        if not new_inferred:
            return False

# Run
if forward_chaining(rules, facts, goal):
    print("Goal reached!")
else:
    print("Goal not reachable.")

```

... Inferred: C
Inferred: D
Inferred: E
Goal reached!


```

# Rules: IF conditions THEN conclusion
rules = [
    (['A', 'B'], 'C'),
    (['D'], 'A')
]

# Known facts
facts = {'B', 'D'}

def backward_chain(goal, facts, rules, visited=set()):

    print("Trying to prove:", goal)

    # Goal already known
    if goal in facts:
        print(goal, "is a fact")
        return True

    # Avoid loops
    if goal in visited:

```

```

# Define rules
rule1 = ctrl.Rule(temperature['cold'], fan_speed['low'])
rule2 = ctrl.Rule(temperature['warm'], fan_speed['medium'])
rule3 = ctrl.Rule(temperature['hot'], fan_speed['high'])

# Control system
fan_ctrl = ctrl.ControlSystem([rule1, rule2, rule3])
fan_sim = ctrl.ControlSystemSimulation(fan_ctrl)

# Input value
fan_sim.input['temperature'] = 25

# Compute output
fan_sim.compute()

# Output result
print("Fan Speed:", fan_sim.output['fan_speed'])

# Visualize (optional)
temperature.view()
plt.show()
fan_speed.view()
plt.show()

```

```

print(f"Attempting to prove goal: {goal_to_prove}")
if backward_chain(goal_to_prove, facts, rules):
    print(f"Goal '{goal_to_prove}' is provable.")
else:
    print(f"Goal '{goal_to_prove}' is not provable.")

```

```

...
Attempting to prove goal: C
Trying to prove: C
Checking: ['A', 'B'] -> C
Trying to prove: A
Checking: ['D'] -> A
Trying to prove: D
D is a fact
A proved
Trying to prove: B
B is a fact
C proved
Goal 'C' is provable.

```

```

import skfuzzy as fuzz
from skfuzzy import control as ctrl
import matplotlib.pyplot as plt

# Define fuzzy variables
temperature = ctrl.Antecedent(np.arange(0, 41, 1), 'temperature')
fan_speed = ctrl.Consequent(np.arange(0, 101, 1), 'fan_speed')

# Membership functions for temperature
temperature['cold'] = fuzz.trimf(temperature.universe, [0, 0, 20])
temperature['warm'] = fuzz.trimf(temperature.universe, [10, 20, 30])
temperature['hot'] = fuzz.trimf(temperature.universe, [20, 40, 40])

# Membership functions for fan speed
fan_speed['low'] = fuzz.trimf(fan_speed.universe, [0, 0, 50])
fan_speed['medium'] = fuzz.trimf(fan_speed.universe, [25, 50, 75])
fan_speed['high'] = fuzz.trimf(fan_speed.universe, [50, 100, 100])

# Define rules
rule1 = ctrl.Rule(temperature['cold'], fan_speed['low'])
rule2 = ctrl.Rule(temperature['warm'], fan_speed['medium'])
rule3 = ctrl.Rule(temperature['hot'], fan_speed['high'])

```

```

# Define rules
rule1 = ctrl.Rule(temperature['cold'], fan_speed['low'])
rule2 = ctrl.Rule(temperature['warm'], fan_speed['medium'])
rule3 = ctrl.Rule(temperature['hot'], fan_speed['high'])

# Control system
fan_ctrl = ctrl.ControlSystem([rule1, rule2, rule3])
fan_sim = ctrl.ControlSystemSimulation(fan_ctrl)

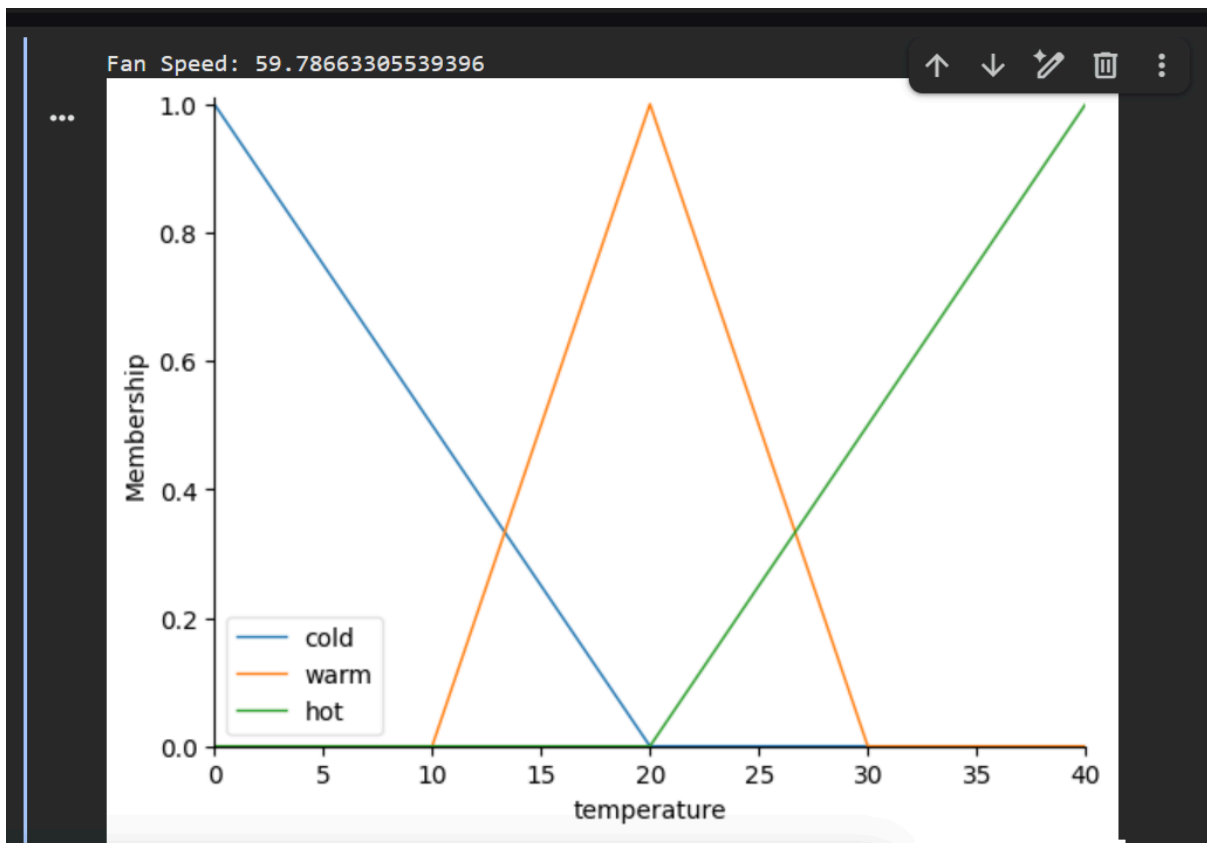
# Input value
fan_sim.input['temperature'] = 25

# Compute output
fan_sim.compute()

# Output result
print("Fan Speed:", fan_sim.output['fan_speed'])

# Visualize (optional)
temperature.view()
plt.show()
fan_speed.view()
plt.show()

```



```
class BlocksWorld:
```

```
    def __init__(self, initial, goal):
        self.initial = initial
        self.goal = goal
```

```
    def is_goal(self, state):
        # A state is a goal if all blocks match the goal configuration
        return state == self.goal
```

```
    def get_top_blocks(self, state):
        # Identify blocks that have nothing on top of them
        on_values = set(state.values()) - {'table'}
        tops = [block for block in state if block not in on_values]
        return tops
```

```
    def move(self, state, block, destination):
        # Simulate moving a block from its current position to a new destination
        new_state = state.copy()
        new_state[block] = destination
        return new_state
```

```

def solve(self):
    # Use Breadth-First Search (BFS) to find a path from initial to goal state
    visited = set()
    queue = deque()

    # Each item in the queue is (current_state, path_to_current_state)
    queue.append((self.initial, []))

    while queue:
        current_state, path = queue.popleft()

        if self.is_goal(current_state):
            return path # Goal reached, return the sequence of moves

        # Convert dictionary state to a frozenset of (key, value) pairs for
        # This makes the state hashable for the 'visited' set
        state_hash = frozenset(current_state.items())

        if state_hash in visited:
            continue

```

```

        if state_hash in visited:
            continue

        visited.add(state_hash)

        # Explore possible moves from the current state
        for move in self.get_possible_moves(current_state):
            new_state = self.move(current_state, move[0], move[1])
            queue.append((new_state, path + [move]))

    return None # Goal not reachable

```

```

}

# Create a BlocksWorld instance
bw_problem = BlocksWorld(initial_state, goal_state)

# Solve the problem
solution = bw_problem.solve()

# Print the solution
if solution:
    print("Solution found:")
    for i, move in enumerate(solution):
        print(f"Step {i+1}: Move block {move[0]} to {move[1]}")
else:
    print("No solution found for the given Blocks World problem.")

```

```

... Solution found:
Step 1: Move block B to C
Step 2: Move block A to B

```

```

▶ print("Grandparent relationships:")
grandparents_of_ann = get_grandparents('thomas', 'ann', parent_facts)
if grandparents_of_ann:
    print(f"Thomas is a grandparent of Ann: {grandparents_of_ann}")
else:
    print("Thomas is not a grandparent of Ann.")

grandparents_of_emma = get_grandparents('thomas', 'emma', parent_facts)
if grandparents_of_emma:
    print(f"Thomas is a grandparent of Emma: {grandparents_of_emma}")
else:
    print("Thomas is not a grandparent of Emma.")

# Example Usage for Sibling:
print("\nSibling relationships:")
siblings_of_ann_david = get_siblings('ann', 'david', parent_facts)
if siblings_of_ann_david:
    print(f"Ann and David are siblings: {siblings_of_ann_david}")
else:
    print("Ann and David are not siblings.")

siblings_of_mary_john = get_siblings('mary', 'john', parent_facts)
if siblings_of_mary_john:
    print(f"Mary and John are siblings: {siblings_of_mary_john}")
else:
    print("Mary and John are not siblings.")

```

```

    print("Ann and David are not siblings.")

siblings_of_mary_john = get_siblings('mary', 'john', parent_facts)
if siblings_of_mary_john:
    print(f"Mary and John are siblings: {siblings_of_mary_john}")
else:
    print("Mary and John are not siblings.")

siblings_of_thomas_mary = get_siblings('thomas', 'mary', parent_facts)
if siblings_of_thomas_mary:
    print(f"Thomas and Mary are siblings: {siblings_of_thomas_mary}")
else:
    print("Thomas and Mary are not siblings.")

*** Grandparent relationships:
Thomas is a grandparent of Ann: [('thomas', 'ann')]
Thomas is a grandparent of Emma: [('thomas', 'emma')]

Sibling relationships:
Ann and David are siblings: [('ann', 'david')]
Mary and John are siblings: [('mary', 'john')]
Thomas and Mary are not siblings.

```